Exam 2 - Results

Attempt 1 of 1

Written Nov 13, 2024 3:33 PM - Nov 13, 2024 4:01 PM

Attempt Score **93.33 %**

Overall Grade (Highest Attempt) **93.33 %**

Question 1 **5 / 5 points**

Which of the following is not touted as a major advantage of functional programming?

- ○ Functional programs can be more efficient, since they are easier to parallelize on multicore or distributed systems
- ○ Functional programs can be more efficient, since previous results of function calls can be safely cached
- ○ Functional programs are easier to prove correct
- ✓ ○ Functional programs are easier to make robust, because instead of throwing errors you can return them instead

Question 2 **5 / 5 points**

Which of the following statements about Java arrays is false?

- ○ They are fixed size (they cannot grow and shrink)
- ○ They are covariant
- ○ They are mutable
- ✓ ○ They can only contain objects, not primitives

Question 3 **5 / 5 points**

In Swift, what percentage of the values in the type `Int64` are also in the type `Int32`? (Choose the closest answer.)

- ◯ 50
- ◯ 200
- ✓ ◯ 0.000000023283064365 39
- ◯ 0.000000023283064359 97
- ◯ 0.000000023283064354 54

Question 4 **5 / 5 points**

Why are `map` and `filter` not used much in Python?

- ✓ ◯ Comprehensions are more Pythonic
- ◯ Python programmers should always write out loops with for-statements
- ◯ No one can remember whether the function is the first argument or the sequence is the first argument
- ◯ They don't work on ranges or other sequences, only lists

Question 5 **5 / 5 points**

What function is this

```
f p [] = []
f p (x :: y) = if p x then x :: f p y else []
```

- ◯ findFirst
- ◯ take
- ◯ dropWhile
- ◯ drop
- ◯ map
- ✓ ◯ takeWhile
- ◯ filter
- ◯ flatMap

Question 6 **5 / 5 points**

Functional programming became popular in the early 21st century thanks to which of the following?

- [x] The consolidation of block-structured languages
- [x] The need for better engines to render HTML in browsers
- [x] The rise of big data
- [x] The rise of ethical AI
- [x] The rise of multicore processors

Question 7 **5 / 5 points**

Which of the following is allowed to appear in a Swift class but not a Swift struct?

- ( ) Weak reference
- ( ) Computed property
- (x) Deinitializer
- ( ) Class method

Question 8 **5 / 5 points**

Unlike many other languages, Swift makes it easy to treat string elements as:

- ( ) Real Unicode characters
- (x) Pretty much actual grapheme clusters
- ( ) UTF-16 code units
- ( ) Bytes

Question 9 **5 / 5 points**

How is the billion dollar mistake manifested in Java?

- ( ) The null pointer exception
- ( ) Dynamic typing mixed with weak typing
- ( ) The null type

- ✓ ◯ The inclusion of null in every reference type

Question 10 **5 / 5 points**

Which of the following is true in Swift?

- ✓ ◯ Instances of value types can be mutable
- ◯ Instances of reference types cannot be immutable
- ◯ Instances of value types are immutable
- ◯ Instances of reference types are mutable

Question 11 **4.167 / 5 points**

Which of the following are equivalent to the Haskell expression

```
(+1).(*2)
```

- ➡ ✓ ☐ `\n -> 2 * n + 1`
- ✓ ☐ `(* 2) $ succ`
- ✓ ☐ `\n -> (n + 1) * 2`
- ✓ ☐ `\n -> succ n * 2`
- ➡ ✗ ☐ `\n -> n + succ n`
- ✓ ☐ `\n -> 2 * (n + 1)`

Question 12 **5 / 5 points**

Which of the following must (always) have its type mentioned explicitly in Java?

- ◯ Lambda expression return type
- ◯ Local variable
- ◯ Lambda expression parameter
- ✓ ◯ Class field

Question 13 **5 / 5 points**

Assuming `Dog` is a subclass of `Animal`, *should* you be able to assign an expression of `List<Dog>` to a variable type constrained as `List<Animal>` in its declaration? (Answer not in terms of what some languages do, but what makes the most sense in terms of type safety.)

- ◯ No! Dog lists are much more powerful than Animal lists
- ◯ Of course! A list of dog IS a list of animals
- ◯ Of course! Dogs are animals and a dog can do anything an animal can do
- ✓ ◯ No! Because this would allow you to put a cat into a list of dogs

Question 14 **5 / 5 points**

Which of the following TypeScript interfaces are reasonable for ternary trees?

✓ ☐

JavaScript

```
interface Node<T> {
  item: T
  left: Node<T>?
  center: Node<T>?
  right: Node<T>?
}
```

✓ ☐

JavaScript

```
interface Node<T> {
  item: T
  left?: Node<T>
  center?: Node<T>
  right?: Node<T>
}
```

✓ ☐

JavaScript

```
interface Node<T> {
  item: T
  left: Node<T>
  center: Node<T>
  right: Node<T>
}
```

✓ ☐

JavaScript

```
interface Node<T> {
  item: T
  children: [Node<T>?, Node<T>?, Node<T>?]
}
```

- [x]

```JavaScript
interface Node<T> {
  item: T
  children: Node<T>[]
}
```

Question 15 **5 / 5 points**

Which statements are true about slices?

- [x] They are backed by arrays
- [x] They are always mutable
- [x] Their contents are accessed through a reference
- [x] They must be allocated on the heap
- [x] They may never be shared

Question 16 **5 / 5 points**

Which of the following are accurate translations, in Haskell, of the Python expression `[x]*y` ?

- [x] `take y . repeat x`
- [x] `take x $ repeat . y`
- [x] `[x .. y]`
- [x] `take y . repeat $ x`
- [x] `[x] `repeat` y`
- [x] `take x >>> repeat y`

Question 17 **4.167 / 5 points**

Which of the following does Alan Kay consider essential to object-orientation?

- [x] Inheritance
- [x] Dynamic Polymorphism
- [x] Encapsulation and information hiding

- [ ] Late binding
- [x] Messaging
- [x] public/private access modifiers

Question 18 **5 / 5 points**

Which of the following can cause of dangling pointers?

- [x] Manually freeing memory prematurely
- [x] The malloc function in C
- [x] Returning a pointer to a local variable
- [x] Not freeing memory
- [x] Putting global variables in the heap

Question 19 **0 / 5 points**

Given the Haskell definition

f x y z = (x == y) && (z < z)

what is the type of f?

- ( )
```Haskell
a * a * Int -> a
```
- (x)
```Haskell
(Ord a) => a -> a -> a -> Bool
```
- ( )
```Haskell
(Eq a, Num b) =>; a -> a -> b -> Bool
```
- ( ) It cannot be given a type because the definition is illegal
- ( )
```Haskell
(Ord c, Eq b) => b -> b -> c -> Bool
```

Question 20 **5 / 5 points**

Which of the following are true about optionals?

- [x] The operations `map`, `filter`, and `flatMap` are perfectly sensible on optionals.
- [x] Java's introduction of optionals completely freed the language of the Billion Dollar mistake
- [x] TypeScript doesn't natively have optionals, but they can be approximated by a union type with one component being the literal type `undefined`.
- [x] Optionals are sum types
- [x] Haskell's optional type is called `Maybe`
- [x] Optionals are product types

Done